



MEMORIA CUARTA PRACTICA: PROCESADOR DE LENGUAJE EN HASKELL

Acedo Blanco, Alvaro
Czepiel Babiarz, Daniel

3 de junio de 2026

Índice

1. ¿Qué es un procesador?	2
1.1. Objetivo de la práctica	2
1.2. Módulos de un procesador	2
1.3. Ámbito de la práctica	2
1.4. Características del lenguaje	3
2. Arquitectura y Decisiones de Diseño	4
2.1. Estructura de módulos	4
2.2. Gestión del estado sin mutabilidad	4
2.3. Tipos de Datos Algebraicos y Pattern Matching	4
2.4. Uso de la librería <code>System.IO</code>	4
2.5. Procesamiento de flujos mediante Entrada/Salida perezosa	5
3. Diseño del analizador léxico	6
3.1. Tokens	6
3.2. Gramática	6
3.3. Diagrama del autómata y acciones semánticas	7
3.4. Descripción de las Acciones Semánticas	7
3.5. Errores	8
4. Diseño del analizador sintáctico	9
4.1. Gramática	9
4.2. Tabla	10
4.3. Implementación Funcional del Análisis Descendente	10
4.4. Gestión de Errores Sintácticos	10
4.4.1. Estrategia de Recuperación	10
5. Diseño del gestor de errores	12
6. Anexo: ficheros de prueba	13
6.1. Ficheros correctos	13
6.1.1. Fichero 1	13
6.1.2. Fichero 2	13
6.1.3. Fichero 3	13
6.1.4. Fichero 4	13
6.1.5. Fichero 5	14
6.2. Ficheros con errores	14
6.2.1. Fichero 1	14
6.2.2. Fichero 2	14
6.2.3. Fichero 3	15
6.2.4. Fichero 4	15
6.2.5. Fichero 5	15

1. ¿Qué es un procesador?

1.1. Objetivo de la práctica

Un procesador se encarga de analizar un fichero que debe de estar escrito en un determinado lenguaje formal, como Java, C, ... (aunque no tiene que ser un lenguaje de programación). Recibe como entrada un fichero a procesar y devuelve otra representación del fichero, más adecuada para el proposito que se persigue, o correcto; o si el fichero contiene errores, un mensaje de error claro que permite corregir el fichero

En el mundo de los lenguajes de programación, es un paso previo a la generación del código objeto para hacer un ejecutable del código.

1.2. Módulos de un procesador

Un procesador se puede dividir principalmente en los siguientes módulos interconectados, los cuales suelen actuar en forma de *pipeline* o cascada, donde la salida de una fase sirve como entrada directa para la siguiente:

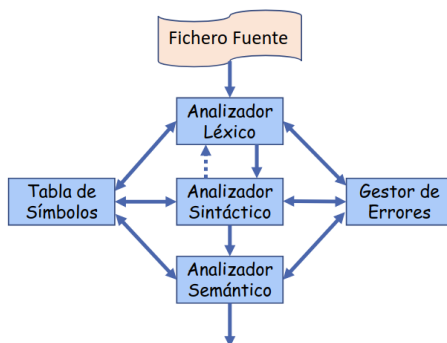


Figura 1: Esquema del procesador.

1. **Analizador léxico (Scanner):** Es la primera fase del proceso. Su objetivo es leer el fichero de entrada carácter a carácter y agruparlos en unidades indivisibles con significado léxico, denominadas *tokens* (como palabras reservadas, identificadores, números, operadores o símbolos de puntuación). Además, se encarga de "limpiar el código", descartando elementos innecesarios para la evaluación lógica, como los espacios en blanco, tabulaciones y comentarios. Internamente, el reconocimiento de estos *tokens* se suele modelar mediante expresiones regulares y Autómatas Finitos Deterministas (AFD).
2. **Analizador sintáctico (Parser):** Recibe el flujo de *tokens* generado por la fase anterior y verifica que su disposición cumple con las reglas gramaticales del lenguaje formal. Su resultado suele ser una representación estructurada, como un Árbol de Sintaxis Abstracta (AST), que refleja la jerarquía de las operaciones.
3. **Analizador semántico:** Toma la estructura generada por el analizador sintáctico y comprueba que, además de estar bien formada, tiene sentido y coherencia. Aquí se realizan tareas como la comprobación de tipos, la verificación del ámbito de las variables o asegurar que una variable ha sido declarada antes de usarse.
4. **Tabla de símbolos:** Es una estructura de datos transversal al resto de analizadores. Actúa como un diccionario donde se registra información vital sobre los identificadores encontrados en el código (variables, funciones, etc.), guardando atributos como su tipo de dato, su ámbito de visibilidad o su posición de memoria.
5. **Gestor de errores:** Es el módulo encargado de interceptar cualquier anomalía durante el proceso de compilación o interpretación. Su misión es recuperarse en la medida de lo posible y emitir mensajes de diagnóstico precisos, indicando la línea y la naturaleza del error para que el programador pueda solucionarlo.

1.3. Ámbito de la práctica

El desarrollo de esta práctica se centra en la implementación de un procesador utilizando el paradigma de programación funcional, concretamente mediante el lenguaje *Haskell* dado en la asignatura. El objetivo principal es construir las fases iniciales descritas anteriormente: análisis léxico y sintáctico; haciendo especial hincapié en el modelado modular.

Se querría haber desarrollado también el analizador semántico sin embargo, debido al tiempo dado para hacer la práctica y su coincidencia con otros trabajos y exámenes nos ha sido imposible terminar la realización de dicho módulo. No obstante, gracias al diseño estrictamente modular empleado en la arquitectura del sistema, se deja la estructura totalmente preparada y abierta para la incorporación futura de esta fase de análisis sin necesidad de reestructurar el núcleo del procesador ya desarrollado.

1.4. Características del lenguaje

Se ha decidido que el procesador realizado se encargará de procesar un lenguaje inventado basado en JavaScript, que tiene características simplificadas para poder hacer la práctica. En la realidad el lenguaje sería más complejo aunque, esto sólo supondría añadir más estados a los autómatas hechos ya que, la lógica subyacente sería la misma.

Generalidades y formato: el lenguaje implementado es *case-sensitive*, esto es que distingue entre mayúsculas y minúsculas; en el que los espacios en blanco, tabulaciones y saltos de línea se tratan como separadores ignorados por el analizador. Las sentencias simples y las declaraciones explícitas finalizan obligatoriamente con un punto y coma (;). El lenguaje admite comentarios de línea mediante el uso del doble operador de división (//).

Estructura de bloques y ámbito: El lenguaje presenta una estructura de bloques delimitados por llaves ({ y }). Maneja los conceptos de:

- **Ámbito global:** Identificadores declarados fuera de cualquier función o variables utilizadas de forma implícita (sin declaración previa), las cuales se asumen por defecto como globales y de tipo entero (`int`).
- **Ámbito local:** Variables declaradas explícitamente mediante la palabra clave `let` en el interior de una función, siendo visibles únicamente desde su declaración hasta el cierre de la misma. No se permite la definición de funciones anidadas.

Tipos de datos y constantes: Se da soporte a los siguientes tipos de datos básicos, sin existencia de conversiones automáticas de tipo implícitas:

- **Entero (`int`):** Representa valores numéricos de 16 bits con signo (rango hasta 32767).
- **Real (`float`):** Valores numéricos de 32 bits con signo. Su representación requiere obligatoriamente al menos un dígito antes y después del punto decimal (ej. 3.0).
- **Cadena (`string`):** Secuencias de caracteres delimitadas por comillas. Ocupan un tamaño fijo en memoria y soportan hasta un máximo de 64 caracteres.
- **Lógico (`boolean`):** Representado mediante las constantes reservadas `true` y `false`.
- **Vacío (`void`):** Utilizado exclusivamente para denotar la ausencia de tipo de retorno o de parámetros en las funciones.

Operadores y precedencia: El sistema modela un conjunto cerrado de operadores agrupados por su funcionalidad (de menor a mayor precedencia):

1. Asignación simple (=).
2. Operador lógico AND (&&).
3. Operadores relacionales de comparación: mayor (>) y mayor o igual (>=).
4. Operadores aritméticos binarios: suma (+) y resta (-).
5. Operadores unarios de alta precedencia: negación lógica (!), autodecremento (-), así como el signo más (+) y menos (-) unarios.

Se permite la utilización de paréntesis de apertura y cierre para alterar el orden natural de evaluación de las expresiones.

Sentencias del lenguaje: Además de las expresiones de asignación, el flujo de ejecución se compone de:

- **Instrucciones de Entrada/Salida:** `read` (para la lectura de datos por teclado hacia una variable) y `write` (para la evaluación y salida por pantalla de expresiones).
- **Estructuras condicionales:** Sentencias de control `if` (sencilla) e `if-else` (compuesta) para la bifurcación del flujo lógico orientada a bloques. Cabe destacar que este dialecto prescinde de bucles iterativos.

Funciones y recursividad: Las funciones se declaran mediante la palabra clave `function` y deben definirse en el ámbito global antes de ser invocadas. El paso de parámetros se realiza estrictamente por valor y se garantiza soporte completo para la recursividad, permitiendo que una función se invoque a sí misma. El retorno de valores se gestiona explícitamente mediante la sentencia `return`.

2. Arquitectura y Decisiones de Diseño

2.1. Estructura de módulos

El código se ha estructurado en módulos con alta coherencia y baja acoplación, separando estrictamente la lógica pura de las mónadas (*IO*):

- **Main.hs**: Actúa como orquestador. Gestiona los argumentos de entrada, la lectura perezosa (*lazy*) del fichero fuente y la apertura/cierre de los ficheros de salida. Es el punto de entrada de la mónada *IO*.
- **Token.hs y Simbolos.hs**: Contienen el núcleo del modelo de datos. Definen los Tipos de Datos Algebraicos (ADTs) que representan los componentes léxicos y gramaticales (terminales y no terminales).
- **Alex.hs**: Implementa el analizador léxico. En lugar de usar bucles imperativos, modela las transiciones del autómata mediante recursión y *Pattern Matching* sobre la cabeza de la cadena de entrada.
- **TablaLL.hs**: Modela la matriz predictiva del analizador sintáctico. Aprovechando el *Pattern Matching* bidimensional de Haskell, mapea pares de (*NoTerminal*, *Token*) a su correspondiente *Regla* gramatical de forma declarativa.
- **Asin.hs**: El analizador sintáctico principal. Gestiona la pila de símbolos y coordina el flujo de análisis, escribiendo de forma secuencial en los ficheros de salida a medida que se validan las derivaciones.
- **GErrores.hs y TablaSimbolos.hs**: Módulos de soporte que definen las estructuras inmutables para el control del estado transversal del procesador.

2.2. Gestión del estado sin mutabilidad

En programación imperativa, estructuras como la Tabla de Símbolos o el Gestor de Errores suelen ser variables globales modificables. En este diseño puramente funcional, la mutabilidad no existe.

El estado se propaga explícitamente a través de las funciones. Por ejemplo, en **Alex.hs**, cada transición de estado del autómata devuelve una tupla (*Resultado*) que no solo contiene el *Token* reconocido, sino también las versiones actualizadas de *GError* y *TablaSimbolos*:

```
1 type Resultado = (Maybe Token, String, GError, TablaSimbolos, TsMod)
2 getToken :: String -> GError -> TablaSimbolos -> Resultado
```

Firma del Analizador Léxico

Este enfoque garantiza que el estado sea determinista y elimina por completo los efectos secundarios impredecibles, facilitando enormemente la depuración y garantizando la integridad de los datos durante todo el ciclo del *pipeline*.

2.3. Tipos de Datos Algebraicos y Pattern Matching

Haskell permite que el código sea prácticamente una transcripción literal de la especificación matemática. La gramática y los tokens se han modelado utilizando tipos suma. Esto otorga al procesador seguridad a nivel de tipos (*Type Safety*):

```
1 data Simbolo
2   = Terminal Token
3   | NoTerminal NoTerminal
4   | Dollar
```

Fragmento de Simbolos.hs

Gracias a esta estructura, el compilador de Haskell, mediante alertas de advertencia (*exhaustiveness checking*), nos obliga a cubrir todos los casos posibles al procesar símbolos, previniendo errores en tiempo de ejecución.

2.4. Uso de la librería System.IO

La interacción con el sistema de archivos del sistema operativo se delega en las funciones de la librería estándar *System.IO*. Se utilizan manejadores de archivos (*Handle*) y modos de apertura explícitos (*WriteMode*) para volcar la información de forma estructurada en tres canales independientes:

- **output/Tokens.txt**: El registro secuencial de los componentes léxicos procesados.
- **output/TablaSimbolos.txt**: El volcado estático de las variables e identificadores únicos.
- **output/Parse.txt**: El conjunto ordenado de índices de reglas que componen la derivación sintáctica.

2.5. Procesamiento de flujos mediante Entrada/Salida perezosa

Una de las decisiones de diseño más relevantes del proyecto radica en la carga del fichero fuente a través de la función `readFile`. En lenguajes imperativos tradicionales como C, la lectura de un fichero suele requerir la asignación previa de un *buffer* de memoria del tamaño total del archivo o bucles iterativos que bloquean el hilo de ejecución de forma activa.

En este procesador, gracias a la **evaluación perezosa** (*lazy*) intrínseca de Haskell, `readFile` no vuelca de golpe el contenido total del archivo en la memoria RAM del ordenador. En su lugar, transforma el archivo en una cadena de caracteres (`String`) evaluada de manera incremental.

A medida que el analizador léxico (`Alex.hs`) demanda nuevos caracteres para transicionar en el autómata y agrupar el siguiente token, el sistema operativo lee físicamente del disco solo los bytes estrictamente necesarios. Este comportamiento permite que el procesador mantenga un consumo de memoria mínimo y constante, independientemente de la longitud del código fuente a compilar, aislando la lógica del compilador de la gestión manual de flujos de E/S.

El axioma es: S y las reglas de producción son:

$$P = \{S \rightarrow \text{EOF} \mid \text{del } S \mid dA \mid cD \mid _D \mid "E \mid (\mid) \mid + \mid -I \mid > G \mid \\ |! \mid \&H \mid = \mid ; \mid \{ \mid \} \mid , \mid /J\}$$

$$\begin{aligned} A &\rightarrow dA \mid .B \mid \lambda \\ B &\rightarrow dC \\ C &\rightarrow dC \mid \lambda \\ D &\rightarrow cD \mid _D \mid dD \mid \lambda \\ E &\rightarrow c_1E \mid \backslash F \mid " \\ F &\rightarrow nE \mid tE \\ G &\rightarrow = \mid \lambda \\ H &\rightarrow \& \\ I &\rightarrow - \mid \lambda \\ J &\rightarrow /K \\ K &\rightarrow c_2K \mid CRS \mid EOF \end{aligned}$$

Siendo:

$$\begin{aligned} \text{del} &= \{\text{espacio, TAB, CR}\} \\ c_1 &= T \setminus \{\backslash, \text{EOF, TAB, "}\} \\ c_2 &= T \setminus \{\text{EOF, TAB}\} \end{aligned}$$

3.3. Diagrama del autómata y acciones semánticas

A partir de la gramática regular, se diseñó el siguiente Autómata Finito Determinista (AFD):

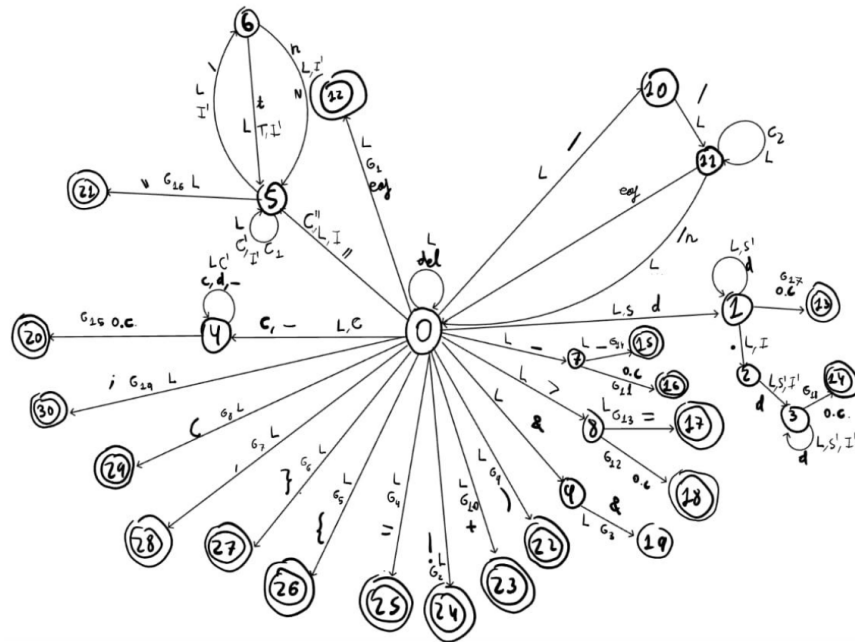


Figura 2: Autómata Finito Determinista del analizador léxico.

3.4. Descripción de las Acciones Semánticas

Conceptualmente, las acciones semánticas del autómata operan actualizando el estado de la lectura. En nuestra implementación en Haskell (*Alex.hs*), estas acciones no mutan variables globales como sí debería hacer en lenguajes como en C u otro lenguaje imperativo, y se podría de hacer mediante el uso de mónadas, sino que se resuelven mediante funciones puras que reciben el estado actual y devuelven el nuevo estado encapsulado.

La lógica abstracta de estas acciones es la siguiente:

G1 a G14, G19: Generación directa de tokens simples sin atributos, por ejemplo, $\langle \text{eof}, - \rangle$, $\langle \text{not}, - \rangle$, $\langle \text{suma}, - \rangle$.

G15 (Identificadores): Se comprueba si el lexema acumulado es una palabra reservada (PR). Si lo es, se genera el token correspondiente. Si no, se consulta la Tabla de Símbolos pura (*insertar0Buscar*): si no existe, se inserta devolviendo la nueva tabla; finalmente, se genera el token $\langle \text{identificador}, \text{pos} \rangle$.

- G16 (Cadenas):** Se valida que el contador de caracteres no exceda el límite (64). Si es válido, se genera $\langle \text{cadena}, \text{lexema} \rangle$; en caso contrario, se emite un error léxico.
- G17 (Enteros):** Se verifica que el valor numérico calculado mediante recursión sea estrictamente menor que $2^{15} - 1$. Si lo supera, se registra un error (**ErrMaxEnt**); si no, se genera $\langle \text{entero}, \text{valor} \rangle$.
- G18 (Reales):** Similar al caso entero, se ensambla la parte entera y decimal mediante paso de parámetros funcionales. Se valida contra la cota máxima permitida para generar $\langle \text{real}, \text{valor} \rangle$ o registrar el error pertinente (**ErrMaxReal**).
- L, C, C', C'', N:** Acciones de lectura y concatenación. En Haskell, este comportamiento está implícito en la recursión sobre la lista de caracteres del archivo fuente, evitando mutaciones de cadenas en memoria.

3.5. Errores

Para la fase léxica, el analizador detecta y gestiona las siguientes anomalías:

- Caracteres desconocidos que no pertenecen al alfabeto del lenguaje.
- El uso incorrecto o inacabado de comentarios (del tipo `//`).
- El uso incompleto del operador lógico **and** (un solo `&`).
- La no terminación lógica de las cadenas de texto antes de un salto de línea.
- Secuencias de escape inválidas dentro de literales de cadena.
- Números reales mal formados (ej. falta de parte decimal tras el punto).
- Exceso de tamaño en los literales numéricos o exceso de longitud en las cadenas.

A diferencia de otros enfoques, cuando el analizador léxico intercepta un error, no se lanzan excepciones que interrumpan el hilo de ejecución. En su lugar, se delega la gestión al módulo **GErrores.hs**, invocando a la función **registrarError**. Esta función toma el registro de estado actual (**GError**) y devuelve un nuevo registro que incluye el código de error y el número de línea exacto concatenado a la lista interna de fallos, permitiendo que el análisis y la recuperación de errores continúe.

4. Diseño del analizador sintáctico

4.1. Gramática

Para el desarrollo del analizador sintáctico se ha diseñado una gramática en contexto formalizada para un análisis descendente tabular ($LL(1)$). Esta se define formalmente mediante la siguiente cuádrupla:

El conjunto de símbolos no terminales (V_N) es:

$$V_N = \{A, B, C, D, E, E', F, G, H, I, K, L, O, P, Q, R, R', S, T, U, U', V, W, X, Z\}$$

El conjunto de símbolos terminales (V_T) corresponde a los tokens identificados por el analizador léxico:

$V_T = \{\text{id, entero, real, cadena, true, false, let, int, float, boolean, string, void, if, else, function, read, write, return, \&\&, >, >=, +, -, !, \dots}\}$

El axioma inicial es:

$$\text{Axioma} = P$$

El conjunto completo de producciones (P) se numera secuencialmente a continuación para facilitar la posterior construcción de la tabla de análisis sintáctico:

- | | |
|--|--|
| (1) $E \rightarrow RE'$ | (26) $S \rightarrow \text{id } G;$ |
| (2) $E' \rightarrow \&\&RE'$ | (27) $S \rightarrow \text{write } E;$ |
| (3) $E' \rightarrow \lambda$ | (28) $S \rightarrow \text{read id};$ |
| (4) $R \rightarrow UR'$ | (29) $S \rightarrow \text{return } X;$ |
| (5) $R' \rightarrow > UR'$ | (30) $G \rightarrow = E$ |
| (6) $R' \rightarrow >= UR'$ | (31) $G \rightarrow (L)$ |
| (7) $R' \rightarrow \lambda$ | (32) $L \rightarrow EQ$ |
| (8) $U \rightarrow VU'$ | (33) $L \rightarrow \lambda$ |
| (9) $U' \rightarrow +VU'$ | (34) $Q \rightarrow , EQ$ |
| (10) $U' \rightarrow -VU'$ | (35) $Q \rightarrow \lambda$ |
| (11) $U' \rightarrow \lambda$ | (36) $X \rightarrow E$ |
| (12) $V \rightarrow +W$ | (37) $X \rightarrow \lambda$ |
| (13) $V \rightarrow -W$ | (38) $B \rightarrow \text{if } (E)Z$ |
| (14) $V \rightarrow !W$ | (39) $Z \rightarrow S$ |
| (15) $V \rightarrow --W$ | (40) $Z \rightarrow \{C\}I$ |
| (16) $V \rightarrow W$ | (41) $I \rightarrow \text{else } D$ |
| (17) $W \rightarrow \text{id } O$ | (42) $I \rightarrow \lambda$ |
| (18) $W \rightarrow (E)$ | (43) $D \rightarrow \text{if } (E)\{C\}I$ |
| (19) $W \rightarrow \text{entero}$ | (44) $D \rightarrow \{C\}$ |
| (20) $W \rightarrow \text{real}$ | (45) $B \rightarrow \text{let } T \text{ id};$ |
| (21) $W \rightarrow \text{cadena}$ | (46) $B \rightarrow S$ |
| (22) $W \rightarrow \text{true}$ | (47) $T \rightarrow \text{int}$ |
| (23) $W \rightarrow \text{false}$ | (48) $T \rightarrow \text{float}$ |
| (24) $O \rightarrow (L)$ | (49) $T \rightarrow \text{boolean}$ |
| (25) $O \rightarrow \lambda$ | (50) $T \rightarrow \text{string}$ |
| (51) $F \rightarrow \text{function } H \text{ id } (A)\{C\}$ | |
| (52) $H \rightarrow T$ | |
| (53) $H \rightarrow \text{void}$ | |
| (54) $A \rightarrow T \text{ id } K$ | |
| (55) $A \rightarrow \text{void}$ | |
| (56) $K \rightarrow , T \text{ id } K$ | |
| (57) $K \rightarrow \lambda$ | |
| (58) $C \rightarrow BC$ | |
| (59) $C \rightarrow \lambda$ | |
| (60) $P \rightarrow BP$ | |
| (61) $P \rightarrow FP$ | |
| (62) $P \rightarrow \lambda$ | |

4.2. Tabla

Se adjunta en la última página de esta sección la tabla del analizador sintáctico. Por motivos de su tamaño, está en su propia página y dada la vuelta para poder caber mejor. Se tuvo que calcular el *First* y el *Follow* de las reglas para poder hacerla, y la tabla demuestra que la gramática es $LL(1)$, ya que, de no serlo, deberían de coincidir en la misma casilla dos o más reglas.

4.3. Implementación Funcional del Análisis Descendente

A diferencia de una implementación imperativa clásica, donde el analizador sintáctico dependería de una pila de memoria mutada explícitamente y una matriz bidimensional dispersa para la tabla predictiva, en este proyecto se ha optado por un enfoque funcional:

- **La Tabla $LL(1)$ como Función Pura (`TablaLL.hs`):** En lugar de reservar memoria estática o dinámica para una matriz de dimensiones $|V_N| \times |V_T|$ que tiene un alto porcentaje de celdas vacías, la tabla se ha modelado como una función pura:

```
1  tablaLL :: NoTerminal -> Token -> Maybe Regla
2
```

Firma de la Tabla $LL(1)$

Aprovechando el *Pattern Matching* de Haskell, solo se definen explícitamente las intersecciones válidas. Cualquier combinación errónea o no contemplada cae en un caso por defecto que devuelve `Nothing`, simulando una celda vacía.

- **Pila de Símbolos y Recursión (`Asin.hs`):** La pila de derivación se modela mediante una lista inmutable de Haskell (`[Símbolo]`), la cual se inicializa con el axioma de la gramática y el símbolo de fin de pila (`[NoTerminal P, Dollar]`). La función principal `bucleAsin` evalúa recursivamente la cabeza de esta lista (la cima de la pila):
 - Si la cima es un **Terminal**, verifica que coincida con el token actual y avanza la lectura.
 - Si la cima es un **No Terminal**, consulta `tablaLL`. Si obtiene una regla, inyecta la lista de símbolos del consecuente al inicio de la pila remanente (`cons ++ resto`) mediante una nueva llamada recursiva, escribiendo simultáneamente el número de regla en el fichero de salida `Parse.txt` aprovechando la mónada `IO`.

4.4. Gestión de Errores Sintácticos

El analizador sintáctico es capaz de detectar y recuperarse de divergencias gramaticales sin abortar la ejecución del compilador. Se interceptan dos naturalezas principales de error:

1. **Discordancia de Terminales:** Ocurre cuando el símbolo en la cima de la pila es un terminal, pero no coincide con el token suministrado por el analizador léxico. El gestor emite un error detallado (ej. *"Se esperaba leer un punto y coma, pero se leyó la palabra reservada if"*).
2. **Celdas Vacías en la Tabla Predictiva:** Ocurre cuando el símbolo en la cima es un No Terminal, pero la `tablaLL` devuelve `Nothing` para el token actual. El módulo `Símbolos.hs` mapea este fallo a un mensaje semántico contextualizado (ej. para el No Terminal F: *"Se esperaba leer la declaración de una función"*).

4.4.1. Estrategia de Recuperación

Frente a la detección de cualquier anomalía sintáctica, la arquitectura funcional permite una recuperación robusta y controlada. Cuando la función `bucleAsin` detecta un error, delega la actualización del registro en la función `registrarErrorAsin` (que añade la falla al historial de la estructura inmutable `GError`).

Tras actualizar el estado, en lugar de arrastrar un árbol sintáctico corrupto, el proceso se resetea recursivamente llamando de nuevo a la función de enrutamiento principal (`parsear`). Esto limpia la pila de símbolos actual, la reinicia a su estado original (`pilaInicial`) y fuerza al analizador a buscar el siguiente punto de sincronización lógico en el flujo de tokens, permitiendo volcar todos los errores del fichero en una sola pasada.

Cuadro 1: Tabla de Análisis Sintáctico

[illegible]

5. Diseño del gestor de errores

Durante los anteriores apartados ya se ha hablado de los errores que se detectan y que mensaje de error se saca en estos casos, por lo tanto, esta sección se va a mencionar ligeramente parte de la implementación.

Para el gestor de errores hemos decidido hacer recuperación de errores, así el gestor recopila los errores del archivo entero y luego los vuelca una vez terminado de analizar el fichero entero. Esto tiene un punto malo y es que si bien se detectan más errores, una parte de los errores detectados es debido al error anterior, por ejemplo, si tenemos una cadena no cerrada, se va a detectar todo lo que este en su misma línea como parte de la cadena, pudiendo dar un error de que no se finaliza con punto y coma. Esto no es un error de diseño sino una decisión consciente, la mayoría de compiladores y lsp (language servers) modernos actúan así para poder ayudar más al programador con el contra de dar más falsos errores.

Para el conteo del número de línea, creamos una función la cual el analizador léxico llama cada vez que lee el delimitador salto de línea. Además, como se ha decidido hacer una implementación que fuera lo más funcional posible, el gestor de errores no es una estructura mutable sino, que cada vez que se actualiza se pasa una copia con el nuevo error o la línea actual actualizada.

6. Anexo: ficheros de prueba

6.1. Ficheros correctos

6.1.1. Fichero 1

```
1  let int a ;
2  let float b ;
3  let boolean bbb ;
4  a = a - c;
5  write a ;
6  write b;
7  write ccc;
```

fichero1.txt

Da un error de memoria el procesador al intentar ejecutar el fichero

TablaTokens:

TablaSimbolos:

Parse:

6.1.2. Fichero 2

```
1  let int x; function float hola (int hola){}
2  hola(x);
3  function void hola2(float f, boolean b){
4      let int i;
5      hola(--i);
6      hola(-i);
7      return ;
8  }
9
10 let boolean y;
11 hola2(hola(x),y);
```

fichero2.txt

Da un error de memoria el procesador al intentar ejecutar el fichero

TablaTokens:

TablaSimbolos:

Parse:

6.1.3. Fichero 3

```
1  function string hello(void){
2      let string hola = "hola";
3      write hola;
4      if(true){
5          return hola;
6      }
7      else{
8          return hello();
9      }
10 }
```

fichero3.txt

Da un error de memoria el procesador al intentar ejecutar el fichero

TablaTokens:

TablaSimbolos:

Parse:

6.1.4. Fichero 4

```
1  let boolean a;
2  let boolean b;
3  a = 3 &&( 4 > ( 2 + 1 ) )&& !true;
4  b=a;
```

```

5 let float rrr;
6 let float ccc; if (rrr >= ccc) b = false;

```

fichero4.txt

TablaTokens:
 TablaSimbolos:
 Arbol de derivación: PP

```

      P
      B
        let
          T
            boolean
            id
            ;
      P
      B
        let
          T
            boolean
            id
            ;
      P
      B
      P

```

6.1.5. Fichero 5

```

1 //f y g no declarados antes, luego son tipo entero
2 write (f - g);
3 if(false){
4     hello();
5 }
6 else if(true){
7     nada = 4;
8 }
9 else{}
10 if (true) read nada;

```

fichero5.txt

TablaTokens:
 TablaSimbolos:
 Parse:

6.2. Ficheros con errores

6.2.1. Fichero 1

```

1 //Creo que falta poner el void
2 function float a fun (){
3 let string tab = 9;
4 return c;
5 }
6 (tab);

```

fichero1.txt

Errores:

6.2.2. Fichero 2

```

1 function int h (int a, float b, string c){
2     if(true){
3         --c;
4         return c;

```

```

5   }
6   else{
7       return h(c, b, a);
8   }
9 }

```

fichero2.txt

Errores:

6.2.3. Fichero 3

```

1 function string hola(float a,b){}
2 pues__nada = hola(a,b);
3 let float a;
4 let int b;
5 if(a < b){}
6 else if(!(a-b)){}

```

fichero3.txt

Errores:

6.2.4. Fichero 4

```

1 if(a){
2     return a;
3 }
4 if(3>2>1) then pues__nada = true;
5 let int b;
6 let float c;
7 a=b+c;

```

fichero4.txt

Errores:

6.2.5. Fichero 5

```

1 function int entero (int a, float b){
2     let string c;
3     return c;
4     return a-b;
5     return ;
6 }

```

fichero5.txt

Errores: